Минобрнауки России

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Волгоградский государственный технический университет»

Факультет _электроники и вычислительной техники_____

Кафедра «Программное обеспечение автоматизированных систем»____

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к контрольной работе

по дисциплине «Теория формальных языков и методов трансляции»_____

на тему «Разработка компилятора для языка программирования Python»_____

Студент_____Чернобаев Егор Сергеевич_____
(фамилия, имя, отчество)

Группа___ПрИн-466_____

Студент_____Теликов Кирилл Евгеньевич_____
(фамилия, имя, отчество)

Группа___ПрИн-466_____

Руководитель работы (проекта) _____ _____Сычев О. А.___
(подпись и дата подписания) (инициалы и фамилия)

Волгоград 2025 г.

Содержание

1 Синтаксис и семантика 3

2 Пример программы на языке Python 4

3 Карточка курсовой работы 6

4 Таблица лексем 8

5 Грамматика языка 11

6 Перечень таблиц, генерируемых на этапе семантического анализа с описаниями структур, описывающих строчки таблицы 14

7 Перечень ошибок, определяемых на этапе семантического анализа 21

Выводы 24

1 Синтаксис и семантика

Python — это интерпретируемый язык программирования с динамической типизацией, поддерживающий множество встроенных типов данных, таких как целые числа, строки и списки. Он обладает лаконичным синтаксисом, что упрощает его изучение и использование.

В Python можно работать с различными встроенными типами данных, такими как:

1) Целые числа (int)
2) Числа с плавающей точкой (float)
3) Строки (str)
4) Списки (list) для одномерных и многомерных структур данных
5) Логические значения (bool)

В Python переменные объявляются динамически, то есть не требуется указывать их тип вручную. Переменной можно присвоить значение просто через оператор присваивания:

```
a = 9
b = 1.15
c = “hello world”
d = [1,2,3,[5,7,8]]
e = True
```

Если переменной не присвоено значение, она не существует, так как в Python отсутствуют нулевые значения для необъявленных переменных.

Python поддерживает 4 основных арифметических операции:

1) Сложение (+)
2) Вычитание (-)
3) Умножение (*)
4) Деление (/)

Python предоставляет возможности для работы с развилками и циклами:

```
x = 10
if x > 5:
    print("x больше 5")
else:
```

```
    print("x меньше или равно 5")
for i in range(5):
    print(i)
n = 0
while n < 5:
    print(n)
    n += 1
```

В Python функции определяются с помощью ключевого слова:

```
def add(a, b):
    return a + b
```

Для работы с консолью используются функции input и print:

```
name = input("Введите ваше имя: ")
print("Привет, " + name)
```

2 Пример программы на языке Python

Пример кода на языке Python представлен в виде простой программы:

```
def bubble_sort(arr, length):
    n = length
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                a = arr[j]
                arr[j] = arr[j + 1]
                arr[j + 1] = a
    return arr
array = [1,1,1,1,1,1,1,1]
for i in range(8):
    array[i] = input()
print("Исходный массив:")
print(array)
sorted_array = bubble_sort(array, 8)
print("Отсортированный массив:")
print(sorted_array)
```

```
def fibonacci_recursive(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci_recursive(n - 1)+fibonacci_recursive(n - 2)
for i in range(1, 10):
    print(fibonacci_recursive(i))

a = 5
b = 3.1
print("5 + 3.1 = ")
print(a + b)
def dfs(array, row, col, rows, cols, start_color, new_color):
    if row < 0 or row >= rows or col < 0 or col >= cols:
        return
    if array[row][col] != start_color:
        return
    array[row][col] = new_color
    dfs(array, row + 1, col, rows, cols, start_color, new_color)
    dfs(array, row - 1, col, rows, cols, start_color, new_color)
    dfs(array, row, col + 1, rows, cols, start_color, new_color)
    dfs(array, row, col - 1, rows, cols, start_color, new_color)
def flood_fill(array, x, y, new_color, rows, cols):
    start_color = array[x][y]
    if start_color == new_color:
        return
    dfs(array, x, y, rows, cols, start_color, new_color)
field = [
        [0,0,0,0,0,0,0,0,0,0],
        [0,1,0,1,1,1,1,1,1,0],
        [0,1,0,1,0,0,0,0,1,0],
        [0,1,0,1,0,1,1,0,1,0],
        [0,1,0,1,0,0,1,0,1,0],
        [0,1,0,1,0,0,1,0,1,0],
        [0,1,0,1,1,1,1,0,1,0],
        [0,1,0,0,0,0,0,0,1,0],
        [0,1,1,1,1,1,1,1,1,0],
        [0,0,0,0,0,0,0,0,0,0]
```

```
]
print("Исходное поле:")
for i in range(0, 10):
    print(field[i])
flood_fill(field, 0, 0, 2, 10, 10)
print("Поле после закраски:")
for i in range(0, 10):
    print(field[i])

a = input()
def test():
    print("TEST CALL")
    return True
if a < 10 and test():
    print("and true")

a = input()
b = input()
print(a + b)
print(a * 3)
print(b * 3)
print(a > b)
print(a >= b)
print(a < b)
print(a <= b)
print(a == b)
print(a != b)
```

3 Карточка курсовой работы

Ниже приведен перечень реализованных функций в рамках данной работы (Таблица 1).

Таблица 1. Карточка с возможностями

Возможность	Вес	Особенности в языке	Кто реализует
Локальные переменные встроенных типов данных : ● целые числа (один тип), ● символы ● строки Выражения с использованием локальных переменных, арифметических операций (4 вида), операций сравнения и присваивания	4 min	Динамическая типизация	Чернобаев Е.С.
Одномерные массивы (или контейнеры), операция доступа к элементу массива	3 min	Аналог - list	Теликов К.Е.
Числовые (целые числа), символьные и строковые константы (литералы) с поддержкой всех видов констант и служебных последовательностей символов	2 min		Чернобаев Е.С.
Управляющие структуры: развилки	3 min		Теликов К.Е.
Управляющие структуры: циклы	3 min		Теликов К.Е.
Функции (процедуры)	3 min		Чернобаев Е.С.
Классы/функции для работы с консолью (ввод/вывод базовых типов данных)	2 min		Чернобаев Е.С.
Переменные, константы и операции (арифметические, сравнения, присваивания) для чисел с плавающей точкой	3		Чернобаев Е.С.
Многомерные массивы	3	Аналог - list	Теликов К.Е.
Логические операции (И, ИЛИ, НЕ)	1		Чернобаев Е.С.
Операции над строками, их участие в выражениях	1		Теликов К.Е.

4 Таблица лексем

Таблица 2. Лексемы

Название	Семантическое значение	Описание
INDENT	-	Отступ, который используется для определения вложенности кода.
LT	<	Проверяет, меньше ли значение левого операнда, чем значение правого операнда.
GT	>	Проверяет, больше ли значение левого операнда, чем значение правого операнда.
LE	<=	Проверяет, меньше ли значение левого операнда или равно значению правого операнда.
GE	>=	Проверяет, больше ли значение левого операнда или равно значению правого операнда.
EQ	==	Проверяет, равны ли значения двух операндов или нет.
NE	!=	Проверяет, не равны ли значения двух операндов.
PLUS_ASSIGN	+=	Добавляет к значению в левом операнде значение из правого операнда.
MINUS_ASSIGN	-=	Вычитает из значения левого операнда значение из правого операнда.

Продолжение таблицы 2.

MUL_ASSIGN	*=	Умножает значение из левого операнда на значение из правого операнда.
DIV_ASSIGN	/=	Делит значение из левого операнда на значение из правого операнда.
INT_TYPE	int	Зарезервированное название функции, используемое для преобразования данных в int число.
FLOAT_TYPE	float	Зарезервированное название функции, используемое для преобразования данных в float число.
BOOL_TYPE	bool	Зарезервированное название функции, используемое для преобразования данных в bool значение.
LIST_TYPE	list	Зарезервированное название функции, используемое для преобразования данных в list.
RANGE_TYPE	range	Зарезервированное название функции, используемое для преобразования данных в range.
STR_TYPE	str	Зарезервированное название функции, используемое для преобразования данных в string.
IF	if	Ключевое слово, является началом для объявления развилки.

Продолжение таблицы 2.

ELIF	elif	Ключевое слово, которое совмещает в себе else и if.
ELSE	else	Добавляет блок с действиями, которые следует выполнять, если условие внутри if является ложным.
CLASS	class	Ключевое слово, является началом для объявления класса.
DEF	def	Ключевое слово, является началом для объявления функции.
FOR	for	Ключевое слово, является началом для объявления цикла for.
WHILE	while	Ключевое слово, является началом для объявления цикла while.
FALSE/TRUE	False/True	Bool значения.
AND	and	Логичское И.
OR	or	Логическое ИЛИ.
NOT	not	Логическое отрицание.
RETURN	return	Ключевое слово, используется для возвращения значения из функций.
IN	in	Ключевое слово используемое в объявлении цикла for.
INT_C	-	Константное значение целого числа.
FLOAT_C	-	Константное значение числа с плавающей точкой.
STRING_C	-	Строковое значение.
ID	-	Наименование переменной/класса/функции.

5 Грамматика языка

```
program: programStmtsList
       | newLineList
       ;

newLineList: NEWLINE
           | newLineList NEWLINE
           ;
programStmtsList: topLevelStmt
                | stmt
                | programStmtsList topLevelStmt
                | programStmtsList stmt
                ;
topLevelStmt: funcDef
            | classDef
            ;
stmt: assignStmt NEWLINE
    | returnStmt
    | ifStmt
    | forStmt
    | whileStmt
    | tryStmt
    | expr NEWLINE
    | stmt NEWLINE
    ;
stmtsList: stmt
         | stmtsList stmt
         ;
classElement: funcDef
            | stmt
            ;
classElementsList: classElement
                 | classElementsList classElement
                 ;
suite: NEWLINE INDENT stmtsList DEDENT
     ;
classSuite: NEWLINE INDENT classElementsList DEDENT
          ;
ifStmt: IF expr ':' suite
      | IF expr ':' suite ELSE ':' suite
      | IF expr ':' suite elifStmtList
      | IF expr ':' suite elifStmtList ELSE ':' suite
      ;
ifHeader: IF expr
        ;
ifHeaderList: ifHeader
            | ifHeaderList ifHeader
            ;
ifHeaderListE: ifHeaderList
             | /* empty */
             ;

elifStmtList: ELIF expr ':' suite
            | elifStmtList ELIF expr ':' suite
            ;
forStmt: FOR targetList IN expr ':' suite
       | FOR targetList IN expr ':' suite ELSE ':' suite
```

```
        ;
    forHeader: FOR targetList IN expr
            ;

    forHeaderList: forHeader
                 | forHeaderList forHeader
                 ;
    whileStmt: WHILE expr ':' suite
             | WHILE expr ':' suite ELSE ':' suite
             ;
    tryStmt: TRY ':' suite exceptStmtList
           | TRY ':' suite exceptStmtList ELSE ':' suite
           | TRY ':' suite exceptStmtList FINALLY ':' suite
           | TRY ':' suite exceptStmtList ELSE ':' suite FINALLY ':'
suite
           | TRY ':' suite FINALLY ':' suite
           ;
    exceptStmt: EXCEPT ':' suite
              | EXCEPT expr ':' suite
              | EXCEPT expr AS identifier ':' suite
              ;
    exceptStmtList: exceptStmt
                  | exceptStmtList exceptStmt
                  ;
    funcDef: funcHeader ':' suite
           ;
    funcHeader: DEF identifier '(' funcArgs ')'
              ;
    param: identifier
         | identifier '=' expr
         ;
    paramsList: param
              | paramsList ',' param
              ;
    paramsListE: paramsList
               | paramsList ','
               | /* empty */
               ;
    classDef: CLASS identifier ':' classSuite
            | CLASS identifier '(' identifier ')' ':' classSuite
            ;
    assignStmt: assignStmtTargetAssignList '=' exprList
              | target PLUS_ASSIGN expr
              | target MINUS_ASSIGN expr
              | target MUL_ASSIGN expr
              | target DIV_ASSIGN expr
              ;
    assignStmtTargetAssignList: exprList
                              | exprList ','
                              | assignStmtTargetAssignList '=' exprList
                              | assignStmtTargetAssignList '=' exprList
','
                              ;
    returnStmt: RETURN exprListE NEWLINE
              ;
    expr: expr '+' expr
        | expr '-' expr
        | expr '*' expr
        | expr '/' expr
```

```
    | expr AND expr
    | NOT expr
    | expr '&' expr
    | expr OR expr
    | expr '|' expr
    | expr GT expr
    | expr GE expr
    | expr LT expr
    | expr LE expr
    | expr EQ expr
    | expr NE expr
    | '+' expr %prec UPLUS
    | '-' expr %prec UMINUS
    | identifier ASSIGN_OP expr
    | '(' expr ')'
    | '[' exprListE ']'
    | '[' expr forHeaderList ifHeaderListE ']'
    | LAMBDA paramsListE ':' expr %prec LAMBDA
    | expr '(' funcArgs ')'
    | expr '.' identifier '(' funcArgs ')'
    | INT_C
    | FLOAT_C
    | STRING_C
    | TRUE
    | FALSE
    | target
    ;
exprE: expr
     | /* empty */
     ;
exprList: expr
        | exprList ',' expr
        ;
exprListE: exprList
         | exprList ','
         | /* empty */
         ;
identifier: ID
          | INT_TYPE
          | FLOAT_TYPE
          | LIST_TYPE
          | RANGE_TYPE
          | BOOL_TYPE
          | STR_TYPE
          ;
identifiers: identifier
           | identifiers ',' identifier
           ;
identifiersE: identifiers
            | identifiers ','
            | /* empty */
            ;
target: identifier
      | expr '[' expr ']'
      | expr '[' slicing ']'
      | expr '.' identifier
      ;
targetList: target
          | targetList ',' target
```

```
            ;
slicing: exprE ':' exprE
       | exprE ':' exprE ':' exprE
       ;
namedArgsList: identifier '=' expr
             | namedArgsList ',' identifier '=' expr
             ;
funcArgs: exprList
        | namedArgsList
        | exprList ','
        | namedArgsList ','
        | exprList ',' namedArgsList
        | exprList ',' namedArgsList ','
        | /* empty */
        ;
```

6 Перечень таблиц, генерируемых на этапе семантического анализа с описаниями структур, описывающих строчки таблицы

Таблица 3. Таблицы constant pool

`def foo(a,b,c):`
`  x = a + b + c`
`def foo2(x,y):`
`  if x > y:`
`  x = x * 2`
`  else:`
`  x = y * 2`
`a = 1`
`b = 2`
`c = 2`
`foo(a,b,c)`
`foo2(3,4)`
	`  #1 = Utf8  Code`
	`  #2 = Utf8 __PROGRAM__`
	`  #3 = Class  #2 // __PROGRAM__`
	`  #4 = Utf8  __BASE__`
	`  #5 = Class  #4 // __BASE__`
	`  #6 = Utf8  print`
	`  #7 = Utf8 (L__BASE__;)V`
	`  #8 = NameAndType  #6:#7 // print:(L__BASE__;)V`
	`  #9 = Methodref  #5.#8 // __BASE__.print:(L__BASE__;)V`
	`  #10 = Utf8  ()V`
	`  #11 = NameAndType  #6:#10 // print:()V`
	`  #12 = Methodref  #5.#11 // __BASE__.print:()V`
	`  #13 = Utf8  input`
	`  #14 = Utf8 (L__BASE__;)L__BASE__;`
	`  #15 = NameAndType  #13:#14 // input:(L__BASE__;)L__BASE__;`
	`  #16 = Methodref  #5.#15 // __BASE__.input:(L__BASE__;)L__BASE__;`
	`  #17 = Utf8 ()L__BASE__;`
	`  #18 = NameAndType  #13:#17`

Продолжение таблицы 3.

	// input:()L__BASE__;
	#19 = Methodref #5.#18
	// __BASE__.input:()L__BASE__;
	#20 = Utf8 range
	#21 = Utf8
	(L__BASE__;L__BASE__;)L__BASE__;
	#22 = NameAndType #20:#21
	//
	range:(L__BASE__;L__BASE__;)L__BASE
	__;
	#23 = Methodref #5.#22
	//
	__BASE__.range:(L__BASE__;L__BASE__
	;)L__BASE__;
	#24 = NameAndType #20:#14
	// range:(L__BASE__;)L__BASE__;
	#25 = Methodref #5.#24
	//
	__BASE__.range:(L__BASE__;)L__BASE_
	_;
	#26 = Utf8
	java/lang/Object
	#27 = Class #26
	// java/lang/Object
	#28 = Utf8 main
	#29 = Utf8
	([Ljava/lang/String;)V
	#30 = NameAndType #28:#29
	// main:([Ljava/lang/String;)V
	#31 = Methodref #3.#30
	//
	__PROGRAM__.main:([Ljava/lang/String;)V
	#32 = Utf8 foo
	#33 = Utf8 <init>
	#34 = NameAndType #33:#10
	// "<init>":()V
	#35 = Methodref #5.#34
	// __BASE__."<init>":()V
	#36 = Utf8
	(L__BASE__;L__BASE__;L__BASE__;)V
	#37 = NameAndType #32:#36
	//
	foo:(L__BASE__;L__BASE__;L__BASE__;
	)V
	#38 = Methodref #3.#37
	//
	__PROGRAM__.foo:(L__BASE__;L__BASE_
	_;L__BASE__;)V
	#39 = Utf8 x
	#40 = String #39
	// x
	#41 = Utf8
	(Ljava/lang/String;)V
	#42 = NameAndType #33:#41
	// "<init>":(Ljava/lang/String;)V
	#43 = Methodref #5.#42

Продолжение таблицы 3.

	//__BASE__."<init>":(Ljava/lang/Str
	ing;)V
	#44 = Utf8 a
	#45 = String #44
	// a
	#46 = Utf8 b
	#47 = String #46
	// b
	#48 = Utf8 __add__
	#49 = NameAndType #48:#14
	// __add__:(L__BASE__;)L__BASE__;
	#50 = Methodref #5.#49
	//
	__BASE__.__add__:(L__BASE__;)L__BAS
	E__;
	#51 = Utf8 c
	#52 = String #51
	// c
	#53 = Utf8 foo2
	#54 = Utf8
	(L__BASE__;L__BASE__;)V
	#55 = NameAndType #53:#54
	// foo2:(L__BASE__;L__BASE__;)V
	#56 = Methodref #3.#55
	//
	__PROGRAM__.foo2:(L__BASE__;L__BASE
	__;)V
	#57 = Utf8 y
	#58 = String #57
	// y
	#59 = Utf8
	__greater__
	#60 = NameAndType #59:#14
	//
	__greater__:(L__BASE__;)L__BASE__;
	#61 = Methodref #5.#60
	//
	__BASE__.__greater__:(L__BASE__;)L_
	_BASE__;
	#62 = Integer 2
	#63 = Utf8 (I)V
	#64 = NameAndType #33:#63
	// "<init>":(I)V
	#65 = Methodref #5.#64
	// __BASE__."<init>":(I)V
	#66 = Utf8 __mul__
	#67 = NameAndType #66:#14
	// __mul__:(L__BASE__;)L__BASE__;
	#68 = Methodref #5.#67
	//
	__BASE__.__mul__:(L__BASE__;)L__BAS
	E__;
	#69 = Utf8 __bVal
	#70 = Utf8 Z
	#71 = NameAndType #69:#70
	// __bVal:Z

Продолжение таблицы 3.

	#72 = Fieldref #5.#71 // __BASE__.__bVal:Z #73 = Integer 1 #74 = Integer 3 #75 = Integer 4 #76 = Methodref #3.#34 // __PROGRAM__."<init>":()V

a = [1,2,3]
a[1] = 10
a[0] = a[1]
	#1 = Utf8 Code
	#2 = Utf8
	__PROGRAM__
	#3 = Class #2
	// __PROGRAM__
	#4 = Utf8 __BASE__
	#5 = Class #4
	// __BASE__
	#6 = Utf8 print
	#7 = Utf8
	(L__BASE__;)V
	#8 = NameAndType #6:#7
	// print:(L__BASE__;)V
	#9 = Methodref #5.#8
	// __BASE__.print:(L__BASE__;)V
	#10 = Utf8 ()V
	#11 = NameAndType #6:#10
	// print:()V
	#12 = Methodref #5.#11
	// __BASE__.print:()V
	#13 = Utf8 input
	#14 = Utf8
	(L__BASE__;)L__BASE__;
	#15 = NameAndType #13:#14
	// input:(L__BASE__;)L__BASE__;
	#16 = Methodref #5.#15
	//
	__BASE__.input:(L__BASE__;)L__BASE_
	_;
	#17 = Utf8
	()L__BASE__;
	#18 = NameAndType #13:#17
	// input:()L__BASE__;
	#19 = Methodref #5.#18
	// __BASE__.input:()L__BASE__;
	#20 = Utf8 range
	#21 = Utf8
	(L__BASE__;L__BASE__;)L__BASE__;
	#22 = NameAndType #20:#21
	//
	range:(L__BASE__;L__BASE__;)L__BASE
	__;
	#23 = Methodref #5.#22
	//
	__BASE__.range:(L__BASE__;L__BASE__
	;)L__BASE__;
	#24 = NameAndType #20:#14
	// range:(L__BASE__;)L__BASE__;

Продолжение таблицы 3.

	#25 = Methodref #5.#24 // __BASE__.range:(L__BASE__;)L__BASE__; #26 = Utf8 java/lang/Object #27 = Class #26 // java/lang/Object #28 = Utf8 main #29 = Utf8 ([Ljava/lang/String;)V #30 = NameAndType #28:#29 // main:([Ljava/lang/String;)V #31 = Methodref #3.#30 // __PROGRAM__.main:([Ljava/lang/String;)V #32 = Utf8 a #33 = String #32 // a #34 = Utf8 <init> #35 = Utf8 (Ljava/lang/String;)V #36 = NameAndType #34:#35 // "<init>":(Ljava/lang/String;)V #37 = Methodref #5.#36 // __BASE__."<init>":(Ljava/lang/String;)V #38 = Integer 1 #39 = Utf8 (I)V #40 = NameAndType #34:#39 // "<init>":(I)V #41 = Methodref #5.#40 // __BASE__."<init>":(I)V #42 = Integer 2 #43 = Integer 3 #44 = Utf8 java/util/ArrayList #45 = Class #44 // java/util/ArrayList #46 = Utf8 (Ljava/util/Collection;)V #47 = NameAndType #34:#46 // "<init>":(Ljava/util/Collection;)V #48 = Methodref #45.#47 // java/util/ArrayList."<init>":(Ljava/util/Collection;)V #49 = Utf8 asList #50 = Utf8 ([Ljava/lang/Object;)Ljava/util/List;

Продолжение таблицы 3.

	#51 = NameAndType #49:#50 // asList:([Ljava/lang/Object;)Ljava/util/List; #52 = Utf8 java/util/Arrays #53 = Class #52 // java/util/Arrays #54 = Methodref #53.#51 // java/util/Arrays.asList:([Ljava/lang/Object;)Ljava/util/List; #55 = Utf8 (Ljava/util/ArrayList;)V #56 = NameAndType #34:#55 // "<init>":(Ljava/util/ArrayList;)V #57 = Methodref #5.#56 // __BASE__."<init>":(Ljava/util/ArrayList;)V #58 = Integer 10 #59 = Utf8 __member_access_assign__ #60 = NameAndType #59:#21 // __member_access_assign__:(L__BASE__;L__BASE__;)L__BASE__; #61 = Methodref #5.#60 // __BASE__.__member_access_assign__:(L__BASE__;L__BASE__;)L__BASE__; #62 = Utf8 __member_access__ #63 = NameAndType #62:#14 // __member_access__:(L__BASE__;)L__BASE__; #64 = Methodref #5.#63 // __BASE__.__member_access__:(L__BASE__;)L__BASE__; #65 = Integer 0 #66 = NameAndType #34:#10 // "<init>":()V #67 = Methodref #3.#66 // __PROGRAM__."<init>":()V

```
struct Field {

    int number = -1;      // Номер поля

    int nameNumber = -1; // Номер названия поля

    string name = "";     // Название поля
```

```
    ExprNode* nameNode = nullptr; // Узел с названием
    int descriptorNumber = -1; // Номер дескриптора
    string descriptor = ""; // Дескриптор (тип переменной)
    string className = "__BASE__"; // Название класса
    AccessFlag accessModifier; // Модификатор доступа
};
struct Method {
    int number = -1; // Номер метода
    bool isContainReturn = false;
    // Name : Descriptor
    map<string, string> varType;
    // Для имени метода
    string name = "";
    int nameNumber = -1;
    // Дескриптор нужен для корректной работы JVM
    string descriptor = "";
    int descriptorNumber = -1;
    // Для локальных переменных
    vector<string> localVars = {};
    // Ссылка на узел дерева с элементами тела
    StmtsListNode* suite = nullptr;
    // Ссылка на сам метод
    int selfMethodRef = -1;
    // Количество параметров (передаваемых в функцию)
    int paramsCount = -1;
    // Ссылка на super класс (в моем случае, __BASE__)
    int baseClassNumber = -1;
    // Ссылка на конструктор super класса
    int baseConstructorNumber = -1;
    // Модификатор доступа
    AccessFlag accessModifier;
```

```
    // ИСПОЛЬЗУЕТСЯ ТОЛЬКО В КОНСТРУКТОРАХ КЛАССОВ
    bool isClassCreated = false;
    StmtNode* returnNode = nullptr;
};
class Class {
public:
    int number; // Номер класса
    // Родительский класс
    int parentNumber;
    Class* parent;
    string name; // Название класса
    // Таблицы
    map<Constant, int> constants;
    map<string, Method*> methods;
    map<string, Field*> fields;
    // Название функции, где вызывается + узел вызова
    vector<pair<string, ExprNode*>> funcMethodCalls = {};
    // Название функции, где вызывается + узел с присваиванием
    vector<pair<string, StmtNode*>> assignCalls = {};
    // Название функции, где вызывается + узел с обращением к полю
    vector<pair<string, ExprNode*>> attributeRefs = {};
private:
    long long _ID = 0;
};
```

7 Перечень ошибок, определяемых на этапе семантического анализа

Далее будут представлены ошибки и примеры, в которых эти ошибки возникают.

Неверное количество передаваемых параметров при вызове функции.

Код:

```
def fibonacci_recursive(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci_recursive(n - 1)+fibonacci_recursive(n - 2)
for i in range(1, 10):
    print(fibonacci_recursive(i, 5))
```

Ошибка:

```
S: ERROR -> function "fibonacci_recursive" takes 1 arguments but 2 was given
```

Использование несуществующей переменной.

Код:

```
def test(a):
    print(a)
test(unknownVar)
```

Ошибка:

```
S: ERROR -> variable "unknownVar" is not defined. Function call "test" in method "main" class "__PROGRAM__"
```

Вызов неизвестной функции.

Код:

```
test()
```

Ошибка:

```
S: ERROR -> trying call unknown function "test"
```

Присвоение новой переменной неизвестной переменной.

Код:

```
a = b
```

Ошибка:

```
S: ERROR -> Trying to assign unknown value "b" to variable "a" in method "main" of class "__PROGRAM__"
```

Возвращение из функции неизвестной переменной.

Код:

```
def test():
    return a
test()
```

Ошибка:

```
S: ERROR -> return value "a" is not defined. Function name "test".
```

Вызов в for функции, которая возвращает void:

Код:

```
def test():
    return
for i in test():
    print(i)
```

Ошибка:

```
S: ERROR -> for takes void arg. Class "__PROGRAM__" method "main"
```

8 Перечень преобразований дерева и дополнительной информации для узлов дерева на этапе семантического анализа

На этапе преобразования дерева происходит:

1) замена “[]” и “=” на “[]=”.

На этапе преобразования дерева добавляется следующая информация:

1) проверяется проверка правильность присвоения одному операнду другого;

2) подсчитываю количество передаваемых аргументов при вызове функции.

Выводы

Изучены особенности синтаксиса и семантики динамически типизированного языка Python. Реализованы основные конструкции языка, такие как переменные, управляющие структуры (развилки и циклы), одномерные и многомерные массивы, арифметические и логические операции, функции, а также ввод и вывод данных. Разработаны лексический и синтаксический анализаторы кода. Также реализована генерация байткода с использованием JVM для обеспечения возможности выполнения программ.